todd-5530-m9

AUTHOR
Reagan Todd

This is a demonstration of how to find mismatches in SQL. Created 2026-04-03 by Reagan Todd. The copyright is for educational uses only.

```
R.version.string
```

```
[1] "R version 4.5.2 (2025-10-31)"
```

```
Sys.Date()
```

```
[1] "2026-04-14"
```

```
library(tidyverse)
library(DBI)
library(RSQLite)
library(readr)
```

Use the MIMIC III database.

Q1. Do an inner join of diagnoses_icd and admissions table to find matching diagnoses & admissions. Display the first ten rows of data only.

Use tables diagnoses_icd Diagnosis codes per admission (subject_id, hadm_id) admissions Admission info (subject_id, hadm_id, admittime, etc.) use hadm_id to join the tables and detect mismatches. ## First we ensure out database is set to use

```
con <- dbConnect(SQLite(), "mimic_demo.db")

diagnosis <- read_csv(
  "~/Documents/Spring 26/MEDB-5530/data/mimic-iii-demo/DIAGNOSES_ICD.csv"
)
```

```
Rows: 1761 Columns: 5
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): icd9_code
dbl (4): row_id, subject_id, hadm_id, seq_num
```

```
ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
dbWriteTable(con, "diagnosis", diagnosis, overwrite = TRUE)

admissions <- read_csv(
  "~/Documents/Spring 26/MEDB-5530/data/mimic-iii-demo/ADMISSIONS.csv"
)
```

```
Rows: 129 Columns: 19
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr  (9): admission_type, admission_location, discharge_location, insurance,...
dbl  (5): row_id, subject_id, hadm_id, hospital_expire_flag, has_chartevents...
dttm (5): admittime, dischtime, deathtime, edregtime, edouttime

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
dbWriteTable(con, "admissions", admissions, overwrite = TRUE)
glimpse(admissions)
```

```
Rows: 129
Columns: 19
$ row_id               <dbl> 12258, 12263, 12265, 12269, 12270, 12277, 12278, …
$ subject_id           <dbl> 10006, 10011, 10013, 10017, 10019, 10026, 10027, …
$ hadm_id              <dbl> 142345, 105331, 165520, 199207, 177759, 103770, 1…
$ admittime            <dttm> 2164-10-23 21:09:00, 2126-08-14 22:32:00, 2125-1…
$ dischtime            <dttm> 2164-11-01 17:15:00, 2126-08-28 18:59:00, 2125-1…
$ deathtime            <dttm> NA, 2126-08-28 18:59:00, 2125-10-07 15:13:00, NA…
$ admission_type       <chr> "EMERGENCY", "EMERGENCY", "EMERGENCY", "EMERGENCY…
$ admission_location   <chr> "EMERGENCY ROOM ADMIT", "TRANSFER FROM HOSP/EXTRA…
$ discharge_location   <chr> "HOME HEALTH CARE", "DEAD/EXPIRED", "DEAD/EXPIRED…
$ insurance            <chr> "Medicare", "Private", "Medicare", "Medicare", "M…
$ language             <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, "POLI", NA, N…
$ religion             <chr> "CATHOLIC", "CATHOLIC", "CATHOLIC", "CATHOLIC", "…
$ marital_status       <chr> "SEPARATED", "SINGLE", NA, "DIVORCED", "DIVORCED"…
$ ethnicity            <chr> "BLACK/AFRICAN AMERICAN", "UNKNOWN/NOT SPECIFIED"…
$ edregtime            <dttm> 2164-10-23 16:43:00, NA, NA, 2149-05-26 12:08:00…
$ edouttime            <dttm> 2164-10-23 23:00:00, NA, NA, 2149-05-26 19:45:00…
$ diagnosis            <chr> "SEPSIS", "HEPATITIS B", "SEPSIS", "HUMERAL FRACT…
$ hospital_expire_flag <dbl> 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0…
$ has_chartevents_data <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1…
```

```r
glimpse(diagnosis)
```

```
Rows: 1,761
Columns: 5
$ row_id     <dbl> 112344, 112345, 112346, 112347, 112348, 112349, 112350, 112…
```

```
$ subject_id <dbl> 10006, 10006, 10006, 10006, 10006, 10006, 10006, 10006, 100…
$ hadm_id    <dbl> 142345, 142345, 142345, 142345, 142345, 142345, 142345, 142…
$ seq_num    <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, …
$ icd9_code  <chr> "99591", "99662", "5672", "40391", "42731", "4280", "4241",…
```

Now we can answer the question

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
matches <- dbGetQuery(con, "
SELECT
    d.subject_id,
    d.hadm_id,
    d.icd9_code,
    datetime(a.admittime, 'unixepoch') AS admittime_dt
FROM diagnosis d
INNER JOIN admissions a
    ON d.hadm_id = a.hadm_id;
")

glimpse(matches)
```

```
Rows: 1,761
Columns: 4
$ subject_id   <dbl> 10006, 10006, 10006, 10006, 10006, 10006, 10006, 10006, 1…
$ hadm_id      <dbl> 142345, 142345, 142345, 142345, 142345, 142345, 142345, 1…
$ icd9_code    <chr> "99591", "99662", "5672", "40391", "42731", "4280", "4241…
$ admittime_dt <chr> "2164-10-23 21:09:00", "2164-10-23 21:09:00", "2164-10-23…
```

```
dbDisconnect(cox)
```

Interpretation of the code

Same as Module 6, we have to convert the unixepoch date into a format that we as humans can easily identify, that is why we created the admittime_dt to see exactly what date that patient was admitted.

To find the number of matching records

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
matches_count <- dbGetQuery(con, "
SELECT COUNT(*) AS total_matches
FROM diagnosis d
INNER JOIN admissions a
    ON d.hadm_id = a.hadm_id;
")

matches_count
```

```
  total_matches
1          1761
```

```
dbDisconnect(cox)
```

Interpretation of output

We can see the first 10 rows of the data from the first chunk of code, and using the second we can see that we were able to obtain 1,761 matching records. Since we used an inner join, we can calculate the number of mismatches that the join omitted:

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
mismatch_count1 <- dbGetQuery(con, "
SELECT COUNT(*) AS total_diagnosis FROM diagnosis
")

mismatch_count1
```

```
  total_diagnosis
1            1761
```

```
dbDisconnect(cox)
```

Interpretation of this output

With a row total of 1,761 for the total diagnosis, it looks like there were no mismatches between the diagnosis table and admissions table.

Q2 Write a sql program to select all Diagnoses and Admissions using diagnoses_icd and admissions table. Hint: Use Full Outer join.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
full_outer <- dbGetQuery(con, "
SELECT
    d.*,
    a.*
FROM diagnosis d
LEFT JOIN admissions a
    ON d.hadm_id = a.hadm_id

UNION

SELECT
    d.*,
    a.*
```

```
FROM admissions a
LEFT JOIN diagnosis d
    ON d.hadm_id = a.hadm_id
")

head(full_outer, 10)
```

```
   row_id subject_id hadm_id seq_num icd9_code row_id subject_id hadm_id
1  112344      10006  142345       1     99591  12258      10006  142345
2  112345      10006  142345       2     99662  12258      10006  142345
3  112346      10006  142345       3      5672  12258      10006  142345
4  112347      10006  142345       4     40391  12258      10006  142345
5  112348      10006  142345       5     42731  12258      10006  142345
6  112349      10006  142345       6      4280  12258      10006  142345
7  112350      10006  142345       7      4241  12258      10006  142345
8  112351      10006  142345       8      4240  12258      10006  142345
9  112352      10006  142345       9      2874  12258      10006  142345
10 112353      10006  142345      10     03819  12258      10006  142345
    admittime  dischtime deathtime admission_type   admission_location
1  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
2  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
3  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
4  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
5  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
6  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
7  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
8  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
9  6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
10 6147695340 6148458900        NA      EMERGENCY EMERGENCY ROOM ADMIT
   discharge_location insurance language religion marital_status
1    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
2    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
3    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
4    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
5    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
6    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
7    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
8    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
9    HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
10   HOME HEALTH CARE  Medicare     <NA> CATHOLIC      SEPARATED
                 ethnicity  edregtime  edouttime diagnosis hospital_expire_flag
1  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
2  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
3  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
4  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
5  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
6  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
7  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
8  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
9  BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                     0
```

```
10 BLACK/AFRICAN AMERICAN 6147679380 6147702000    SEPSIS                    0
   has_chartevents_data
1                     1
2                     1
3                     1
4                     1
5                     1
6                     1
7                     1
8                     1
9                     1
10                    1
```

```
dbDisconnect(cox)
```

Interpretation of code

We use the union term here, this term allows us to use two select statements which combines those queries and removes duplicates. This is primarily useful in R Quarto where a SQL query does not allow for a traditional full outer join.

Interpretation of the output

Here we can see that we now have 24 columns (limited to 10 rows) of patient data using a full outer join to join the admissions and diagnosis data sets. We will have 1,761 observations or rows and 24 columns.

Q3 Which join can be used to detect all match and mismatch cases.

Answer to question 3

A full outer join as done in question 2 allows us to detect all matches and mis- matches due to its ability to join all of the data regardless of a match in the next table.

Q4. Which patients(or subject_id) appear in both admissions and diagnoses_icd? Hint use two select statements and the intersect statement.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
intersect <- dbGetQuery(con, "
SELECT DISTINCT subject_id
FROM admissions

INTERSECT
```

```
SELECT DISTINCT subject_id
FROM diagnosis;
")
glimpse(intersect)
```

```
Rows: 100
Columns: 1
$ subject_id <dbl> 10006, 10011, 10013, 10017, 10019, 10026, 10027, 10029, 100…
```

```
dbDisconnect(cox)
```

Interpretation of the code

Here we used the [intersect][ref02] term which allows us to return only the records that were found within both select statements. This essentially shows us what is in the center of a vendiagram.

Interpretation of the output

We have 100 patients who appear both in the diagnosis and admissions data.